

# Sistema de Pasajeros de Hotel - Duerme Bien

Tema 6 · Luis Huenchul

Informe de requerimientos y modelado

## 1. Introducción

### 1.1 Propósito

Definir qué debe hacer el sistema que reemplazará las planillas Excel del hotel Duerme Bien y relacionar requerimientos, casos de uso, procesos, datos e interfaces. Se utilizan las secciones IEEE 830 adaptadas enumeradas en el documento de definición de proyectos.

### 1.2 Alcance

Habitaciones, huéspedes, ocupación y disponibilidad, check-in, check-out, costos por pasajero, reservas, usuarios y los informes de ocupación y reservas. Se conserva el tema 6; no se agregan pagos electrónicos, restaurante, inventario ni otros servicios.

### 1.3 Público objetivo

Administrador y encargados del hotel, equipo de desarrollo y docente.

### 1.4 Definiciones

Huésped: pasajero identificado. Reserva: alojamiento previsto para un período. Estadía: alojamiento efectivamente registrado. Participación: vínculo entre una estadía y uno de sus huéspedes. Check-in: ingreso y asignación. Check-out: cálculo de cuenta y cierre. Wireframe: estructura de pantalla. Mockup: diseño visual. Prototipo: interfaz que permite probar el flujo.

## 2. Descripción general

### 2.1 Perspectiva del producto

El sistema centraliza la información hoy mantenida en Excel. Las habitaciones conservan capacidad y orientación; las reservas y estadías conservan su historial. La disponibilidad se obtiene de los registros vigentes, en lugar de guardar un estado futuro ambiguo en la habitación.

### 2.2 Funciones generales

Registro y edición de habitaciones, registro de huéspedes, consulta de disponibilidad, asignación de huéspedes, cierre de estadías, cálculo individual, reservas, gestión de perfiles e informes.

### 2.3 Clases de usuario

El caso identifica Administrador y Encargado. Para este diseño se propone que el Administrador herede las interacciones operativas del Encargado y además administre habitaciones, usuarios e informes. Es una matriz propuesta, no una respuesta real del hotel. En las clases de dominio se representa un Usuario asociado a un Rol; no se duplican las mismas personas en subclases de usuarios.

### 2.4 Entorno operativo

El prototipo usa HTML, CSS y JavaScript en el navegador, con almacenamiento local. Implementa operaciones y validaciones para demostrar el diseño. La selección de usuario simula una sesión; no autentica identidades reales. El sistema final necesita servidor, base de datos, control transaccional y autenticación segura. El caso no fija motor ni infraestructura.

## 2.5 Restricciones

Mantener capacidad, orientación, asignación de pasajeros, cálculo individual, ambos perfiles e informes. No inventar tarifas ni entrevistas reales. Los ejemplos son ficticios y los importes se expresan en CLP.

## 2.6 Supuestos y dependencias

Se propone una habitación completa por reserva y estadía; cada estadía identifica a todos sus pasajeros. Los intervalos se interpretan como [entrada, salida): una salida permite otra entrada ese día. Compartir habitación entre estadías, reservas de varias habitaciones, permisos exactos y regla de cobro requieren confirmación. El diseño se ajustará si la entrevista define esas condiciones de otra forma.

# 3. Requerimientos

## 3.1 Requerimientos funcionales

| ID   | Requerimiento                        | Criterio observable                                                                                                                              |
|------|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| RF01 | Gestionar habitaciones               | Registrar número único, capacidad entera positiva y orientación. Editar sin invalidar reservas o estadías vigentes.                              |
| RF02 | Registrar e identificar huéspedes    | Guardar identificación única, nombres y apellidos; seleccionar huéspedes existentes para evitar duplicarlos. Asociar cada pasajero a su estadía. |
| RF03 | Consultar ocupación y disponibilidad | Mostrar ocupación actual y habitaciones compatibles con fechas y cantidad de pasajeros. Considerar reservas vigentes y estadías activas.         |
| RF04 | Registrar check-in                   | Permitir ingreso con o sin reserva. Verificar habitación, intervalo y todos los pasajeros; crear estadía activa y asignaciones individuales.     |
| RF05 | Registrar check-out                  | Seleccionar estadía activa, revisar la cuenta y confirmar la salida. Conservar historial y liberar ocupación.                                    |
| RF06 | Calcular costos por pasajero         | Obtener huéspedes de la estadía, aplicar la regla de cobro y mostrar cada costo y total. Conservar los resultados individuales.                  |

| ID   | Requerimiento                 | Criterio observable                                                                                                                          |
|------|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| RF07 | Gestionar usuarios y perfiles | Registrar usuarios, asignar Administrador o Encargado, controlar operaciones por perfil y estado activo.                                     |
| RF08 | Gestionar reservas            | Registrar titular, habitación, fechas y cantidad. Modificar o cancelar sólo reservas registradas; comprobar disponibilidad antes de guardar. |
| RF09 | Generar informes              | Consultar ocupación y reservas para un período; mostrar habitación, fechas, pasajeros y estado; representar también resultados vacíos.       |

### 3.2 Requerimientos no funcionales propuestos

| ID    | Criterio                     | Aceptación                                                                                                                                        |
|-------|------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| RNF01 | Acceso por perfil            | Rechazar una operación restringida aun si se intenta fuera de la navegación visible.                                                              |
| RNF02 | Integridad de datos          | Rechazar duplicados, fechas incoherentes, capacidades inválidas y pasajeros no identificados.                                                     |
| RNF03 | Usabilidad                   | Etiquetas, navegación consistente, detalle de cuenta y mensajes concretos de corrección.                                                          |
| RNF04 | Persistencia                 | Los datos de demostración sobreviven al recargar el navegador; el sistema final usa almacenamiento central.                                       |
| RNF05 | Consistencia concurrente     | El sistema final confirma disponibilidad y guarda asignación en una transacción. El prototipo local no demuestra concurrencia entre dispositivos. |
| RNF06 | Trazabilidad y mantenimiento | Conservar un código único por caso de uso y la relación con requerimientos, datos y pantallas.                                                    |

### 3.3 Matriz de permisos propuesta

| Operación                                 | Encargado | Administrador |
|-------------------------------------------|-----------|---------------|
| Acceso y consulta de disponibilidad       | Sí        | Sí            |
| Huéspedes, reservas, check-in y check-out | Sí        | Sí            |
| Gestión de habitaciones                   | No        | Sí            |
| Gestión de usuarios                       | No        | Sí            |
| Informes de ocupación y reservas          | No        | Sí            |

## 4. Reglas de negocio

| ID   | Regla                                                                                                                  | Origen                                      |
|------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| RN01 | Pasajeros de una estadía $\leq$ capacidad de habitación.                                                               | Derivada del control de capacidad.          |
| RN02 | No confirmar reservas o estadías con intervalos incompatibles en la misma habitación.                                  | Derivada de disponibilidad.                 |
| RN03 | Cada pasajero se registra individualmente en ESTADIA_HUESPED; no se guarda sólo una cantidad.                          | Derivada de registro y costo por pasajero.  |
| RN04 | Check-out incluye calcular cuenta por pasajero y requiere confirmación antes de cerrar.                                | Imagen UML del hotel y diseño del proceso.  |
| RN05 | Reserva REGISTRADA pasa a CHECK_IN al ingresar; al salir pasa a FINALIZADA. CANCELADA deja de bloquear disponibilidad. | Estados propuestos para gestionar reservas. |
| RN06 | Sólo una estadía ACTIVA puede cerrarse. La salida libera ocupación y conserva historial.                               | Proceso de check-out.                       |
| RN07 | Cada usuario tiene un rol válido: Administrador o Encargado.                                                           | Perfiles explícitos del tema 6.             |

| ID   | Regla                                                                              | Origen                                            |
|------|------------------------------------------------------------------------------------|---------------------------------------------------|
| RN08 | Un huésped no se asigna a dos estadías activas simultáneas.                        | Propuesta de integridad de asignación.            |
| RN09 | No borrar reservas canceladas ni estadías finalizadas para liberar una habitación. | Propuesta para conservar los informes históricos. |

## 4.1 Cálculo de costos

El caso exige cálculo automático por pasajero, pero no entrega tarifa ni unidad de cobro. Para demostrar la operación se ingresa una tarifa ficticia por pasajero y noche. Las noches se calculan a partir de las fechas; costo individual = tarifa de prueba x noches y total = suma de los costos individuales. Se conserva la tarifa aplicada y el costo en cada participación; las noches comunes de la cuenta se guardan una sola vez en ESTADIA. Esto no confirma la política real de cobro. No se agregan impuestos, descuentos ni pagos que no estén en el caso.

# 5. Toma de requerimientos

## 5.1 Ejercicio de entrevista simulada

La actividad solicita entrevista simulada. El siguiente registro organiza preguntas y respuestas sustentadas en el caso escrito; no representa una conversación que haya ocurrido con el hotel. Las respuestas que no constan en el caso quedan pendientes.

| Pregunta                                     | Respuesta del cliente simulado basada en el caso                         | Trazabilidad |
|----------------------------------------------|--------------------------------------------------------------------------|--------------|
| ¿Qué se quiere reemplazar?                   | Las planillas Excel usadas para habitaciones y pasajeros.                | Propósito    |
| ¿Qué se registra de habitaciones?            | Capacidad y orientación; se propone un número único para identificarlas. | RF01         |
| ¿Qué se controla de los pasajeros?           | Su registro y asignación a habitaciones, ocupación y costos.             | RF02-RF06    |
| ¿Qué usuarios existen?                       | Administrador y encargados del hotel.                                    | RF07         |
| ¿Qué informes se necesitan?                  | Ocupación y reservas.                                                    | RF09         |
| ¿Qué tarifa y fórmula utiliza el hotel?      | El caso no las define; confirmar antes de implementar el cobro final.    | RF06         |
| ¿Se reserva una habitación completa o camas? | No consta; el diseño propone habitación completa.                        | RF03, RF08   |

| Pregunta                                 | Respuesta del cliente simulado basada en el caso | Trazabilidad |
|------------------------------------------|--------------------------------------------------|--------------|
| ¿Qué permisos exactos tiene cada perfil? | No consta; validar la matriz propuesta.          | RF07         |

## 5.2 Preguntas pendientes

| ID  | Pregunta                                                                | Afecta       |
|-----|-------------------------------------------------------------------------|--------------|
| P01 | ¿Qué tipo de identificación y datos personales son obligatorios?        | RF02         |
| P02 | ¿Tarifa por noche/día? ¿Puede variar por huésped, habitación o período? | RF06         |
| P03 | ¿Habitaciones compartidas o una habitación por estadía?                 | RF03, RF04   |
| P04 | ¿Varias habitaciones por reserva?                                       | RF08         |
| P05 | ¿Cambios de habitación durante la estadía?                              | RF04         |
| P06 | ¿Permisos definitivos por rol?                                          | RF07         |
| P07 | ¿Condiciones definitivas para modificar/cancelar y extender salidas?    | RF05, RF08   |
| P08 | ¿Campos y filtros exactos de informes?                                  | RF09         |
| P09 | ¿Entorno final, respaldo y concurrencia?                                | RNF04, RNF05 |
| P10 | ¿Plantilla original, rúbrica y retroalimentación de Evaluación 1?       | Evaluación 2 |

## 6. Factibilidad

### 6.1 Técnica

El prototipo verifica que los procesos pueden expresarse mediante formularios, reglas y relaciones entre registros. El modelo usa siete entidades relacionales y una tabla de unión para pasajeros. El sistema final requiere transacciones para impedir doble asignación, credenciales seguras y persistencia central. El navegador local permite revisar funcionalidades, pero no sustituye esas capacidades.

## 6.2 De negocio

La propuesta responde al objetivo de centralizar el registro, evitar duplicados y consultar ocupación y reservas. No se calculan costos de implementación, ahorros ni rentabilidad porque el caso no entrega datos económicos. La factibilidad final depende de validar reglas y entorno con el cliente simulado.

## 7. Modelado funcional

### 7.1 Casos de uso

Se presentan dos vistas del mismo sistema: operación y administración. CU01-CU12 mantienen el mismo significado en diagramas, especificaciones y trazabilidad. Las asociaciones actor-caso son continuas y sin flecha. Las inclusiones son discontinuas, con flecha hacia el caso incluido. CU04, CU09 y CU11 incluyen consultar disponibilidad; CU06 incluye calcular cuenta. El Administrador especializa al actor Encargado en la matriz propuesta, mediante triángulo hueco hacia Encargado. No se inventa una extensión sólo para mostrar el símbolo extend. El Huésped es una entidad atendida por los encargados, no un usuario directo del sistema en el caso dado.

#### CU01 - Iniciar sesión

Actor: Administrador y Encargado.

Precondiciones: Usuario registrado y activo.

Flujo principal:

1. Ingresar identificación y credencial.
2. Validar identidad y estado.
3. Recuperar rol y permisos.
4. Abrir sesión y navegación autorizada.

Alternativas y errores:

A1. Credencial incorrecta o usuario inactivo: rechazar el ingreso y permitir corregir.

Postcondición: Sesión activa con el perfil asignado.

Trazabilidad: RF07.

Observación: En el prototipo se selecciona un usuario demo; no se implementa la autenticación productiva.

#### CU02 - Gestionar habitaciones

Actor: Administrador.

Precondiciones: Sesión con permiso de administración.

Flujo principal:

1. Seleccionar registro nuevo o habitación existente.
2. Ingresar número, capacidad y orientación.
3. Validar número único y capacidad.
4. Si es edición, revisar registros vigentes.
5. Guardar y actualizar el listado.

Alternativas y errores:

A1. Datos inválidos o número duplicado: corregir.

A2. Habitación con reserva registrada o estadía activa: no modificar sus características en la propuesta.

Postcondición: Habitación registrada o actualizada.

Trazabilidad: RF01.

Observación: RF01; la restricción de edición es una propuesta para proteger asignaciones existentes.

#### CU03 - Registrar huéspedes

Actor: Encargado; Administrador por herencia de interacciones.

Precondiciones: Sesión con permiso.

Flujo principal:

1. Ingresar identificación, nombres y apellidos.
2. Buscar coincidencia de identificación.
3. Registrar un nuevo huésped.
4. Mostrar el registro para seleccionarlo en reserva o check-in.

Alternativas y errores:

- A1. Identificación ya existente: informar y seleccionar el huésped registrado.
- A2. Datos incompletos: corregir.

Postcondición: Huésped identificado sin duplicación.

Trazabilidad: RF02.

Observación: El tipo de identificación se valida en la entrevista; no se exige RUT sin evidencia del caso.

## **CU04 - Registrar check-in**

Actor: Encargado; Administrador por herencia de interacciones.

Precondiciones: Sesión con permiso; huéspedes registrados. Si se usa reserva, debe estar REGISTRADA.

Flujo principal:

1. Seleccionar reserva o ingreso directo.
2. Definir habitación y fechas o recuperar las de la reserva.
3. Identificar a todos los huéspedes.
4. Verificar fechas, duplicados y ausencia de otra estadía activa de esos huéspedes.
5. Incluir CU05: consultar disponibilidad y capacidad.
6. Confirmar ingreso.
7. Crear ESTADIA ACTIVA y una participación por huésped.
8. Si corresponde, pasar reserva a CHECK\_IN.

Alternativas y errores:

- A1. Reserva cancelada o ya utilizada: seleccionar otra o ingreso directo.
- A2. Faltan pasajeros identificados, fechas inválidas o huésped ya alojado: corregir.
- A3. Capacidad insuficiente o conflicto: elegir otra habitación o intervalo.
- A4. No confirmar: terminar sin guardar.

Postcondición: Estadía activa con sus pasajeros; habitación ocupada.

Trazabilidad: RF02, RF03, RF04.

Observación: La disponibilidad es una validación del flujo, no una precondición asumida como cierta.

## **CU05 - Consultar disponibilidad**

Actor: Encargado; Administrador por herencia de interacciones.

Precondiciones: Sesión con permiso.

Flujo principal:

1. Ingresar entrada, salida y cantidad.
2. Validar intervalo y cantidad entera positiva.
3. Filtrar habitaciones por capacidad.
4. Excluir reservas REGISTRADAS/CHECK\_IN y estadías ACTIVA que se superpongan.
5. Mostrar habitaciones compatibles.

Alternativas y errores:

- A1. Fechas inválidas: corregir.



A2. Sin resultados: mostrar listado vacío y permitir otra consulta.

Postcondición: Disponibilidad consultable para el intervalo.

Trazabilidad: RF03.

Observación: Puede ejecutarse directamente o como inclusión de CU04, CU09 y CU11.

## **CU06 - Registrar check-out**

Actor: Encargado; Administrador por herencia de interacciones.

Precondiciones: Sesión con permiso y estadía ACTIVA.

Flujo principal:

1. Seleccionar estadía y salida real.
2. Recuperar todos sus pasajeros.
3. Incluir CU07: calcular cuenta individual y total.
4. Mostrar detalle de cuenta.
5. Confirmar cuenta y salida.
6. Revisar posibles conflictos al extender la salida.
7. Guardar costos individuales y salida real.
8. Pasar estadía y reserva asociada a FINALIZADA.
9. Liberar ocupación de habitación.

Alternativas y errores:

- A1. Estadía no activa o salida inválida: rechazar.
- A2. Regla o tarifa sin definir: solicitar definición antes de cerrar.
- A3. No confirmar: conservar estadía activa.
- A4. Extensión de salida con conflicto: resolverlo antes de guardar.

Postcondición: Estadía finalizada, cuenta conservada y habitación liberada.

Trazabilidad: RF05, RF06.

Observación: La liberación no elimina reservas futuras ni historial.

## **CU07 - Calcular cuenta por pasajero**

Actor: Invocado por CU06.

Precondiciones: Estadía activa, pasajeros identificados y regla de cobro definida.

Flujo principal:

1. Recuperar período y pasajeros.
2. Obtener parámetros de la regla aprobada.
3. Calcular costo de cada pasajero.
4. Sumar los costos para obtener total.
5. Presentar el detalle para revisión.

Alternativas y errores:

- A1. Regla, tarifa o período inválido: no producir una cuenta confirmable.
- A2. Importe fuera del rango admitido: rechazar el cálculo.

Postcondición: Cuenta calculada, sin finalizar todavía la estadía.

Trazabilidad: RF06.

Observación: En la demostración:  $\text{noches} = \text{salida} - \text{entrada}$ ;  $\text{costo individual} = \text{tarifa de prueba} \times \text{noches}$ . El caso no confirma esta fórmula.

## **CU08 - Gestionar usuarios**

Actor: Administrador.

Precondiciones: Sesión con permiso administrativo.

Flujo principal:

1. Seleccionar nuevo usuario o existente.
2. Ingresar nombre de usuario.
3. Asignar rol Administrador o Encargado.
4. Indicar estado activo en edición.
5. Validar y guardar.

Alternativas y errores:

- A1. Nombre duplicado o rol inexistente: corregir.
- A2. Cambio del propio rol/estado durante sesión: rechazar en la propuesta.

Postcondición: Usuario guardado con un rol válido.

Trazabilidad: RF07.

Observación: Las credenciales seguras pertenecen a la implementación final; el prototipo usa usuarios demo.

## **CU09 - Registrar reserva**

Actor: Encargado; Administrador por herencia de interacciones.

Precondiciones: Sesión con permiso y titular registrado.

Flujo principal:

1. Seleccionar titular y habitación.
2. Ingresar entrada, salida y cantidad.
3. Validar datos.
4. Incluir CU05 para fechas y capacidad.
5. Confirmar.
6. Guardar reserva REGISTRADA y mostrar su número.

Alternativas y errores:

- A1. Fecha, cantidad o titular inválidos: corregir.
- A2. Conflicto de disponibilidad: cambiar selección.
- A3. No confirmar: no crear reserva.

Postcondición: Reserva registrada; ocupación actual de habitación no cambia.

Trazabilidad: RF03, RF08.

Observación: La reserva bloquea el intervalo futuro, no marca la habitación como ocupada físicamente.

## **CU10 - Generar informes**

Actor: Administrador.

Precondiciones: Sesión con permiso y período válido.

Flujo principal:

1. Seleccionar fechas desde/hasta.
2. Consultar estadías que intersectan el período.
3. Mostrar informe de ocupación con pasajeros y estados.
4. Consultar reservas que intersectan el período.
5. Mostrar titular, habitación, fechas y estado.

Alternativas y errores:

- A1. Período inválido: corregir.
- A2. Sin datos: mostrar informe vacío, no un error.

Postcondición: Informes visibles para el período.

Trazabilidad: RF09.

Observación: Las reservas CANCELADAS se conservan en el informe histórico con su estado.

## **CU11 - Modificar reserva**

Actor: Encargado; Administrador por herencia de interacciones.

Precondiciones: Sesión con permiso y reserva REGISTRADA.

Flujo principal:

1. Seleccionar reserva.
2. Cambiar habitación, titular, fechas o cantidad.
3. Validar campos.
4. Incluir CU05 excluyendo la propia reserva del conflicto.
5. Confirmar y guardar cambios.

Alternativas y errores:

- A1. Reserva en otro estado: impedir modificación.
- A2. Capacidad o disponibilidad incompatibles: corregir sin perder la reserva original.
- A3. No confirmar: conservar valores originales.

Postcondición: Reserva actualizada y todavía REGISTRADA.

Trazabilidad: RF03, RF08.

Observación: No se modifica una reserva ya usada para check-in.

## **CU12 - Cancelar reserva**

Actor: Encargado; Administrador por herencia de interacciones.

Precondiciones: Sesión con permiso y reserva REGISTRADA.

Flujo principal:

1. Seleccionar reserva.
2. Solicitar cancelación.
3. Confirmar la decisión.
4. Marcar CANCELADA.
5. Liberar su bloqueo de disponibilidad sin borrar historial.

Alternativas y errores:

- A1. Reserva en otro estado: impedir cancelación.
- A2. No confirmar: conservar reserva registrada.

Postcondición: Reserva cancelada, conservada en el historial.

Trazabilidad: RF08.

Observación: La cancelación no se modela como extend obligatorio ni como borrado de datos.

## **7.2 Procesos**

Los flujos muestran inicio/fin, entradas/salidas, acciones, decisiones con salidas etiquetadas, retornos por error y finalización sin cambios. Se separan registrar reserva, check-in, check-out y modificar/cancelar reserva. Cada decisión tiene dos salidas. Cancelar una operación conserva el registro original; cancelar una reserva cambia su estado, no borra su historial.

# **8. Clases y modelo de datos**

## **8.1 Clases de dominio**

Rol, Usuario, Habitación, Huesped, Reserva, Estadia y EstadiaHuesped muestran nombre, atributos privados y operaciones públicas con tipos. Usuario se asocia a Rol. Estadia compone sus participaciones EstadiaHuesped; el rombo sólido se ubica en Estadia. Huesped existe independientemente de una participación. Las multiplicidades se indican en cada extremo, no como una etiqueta ambigua en el centro. Los actores Administrador/Encargado representan interacciones; los registros de usuarios se almacenan mediante Usuario y Rol.

## 8.2 Diccionario y claves

| Entidad         | Clave primaria           | Referencias y unicidad                                              | Uso                                                |
|-----------------|--------------------------|---------------------------------------------------------------------|----------------------------------------------------|
| ROL             | id_rol                   | nombre único                                                        | Administrador o Encargado.                         |
| USUARIO         | id_usuario               | id_rol -> ROL;<br>nombre_usuario único                              | Perfil, estado y credencial del sistema final.     |
| HABITACION      | id_habitacion            | numero único                                                        | Capacidad positiva y orientación.                  |
| HUESPED         | id_huesped               | identificacion única                                                | Identidad, nombres y apellidos.                    |
| RESERVA         | id_reserva               | id_habitacion -> HABITACION;<br>id_huesped_titular -> HUESPED       | Fechas, cantidad y estado.                         |
| ESTADIA         | id_estadia               | id_habitacion -> HABITACION; id_reserva opcional y único -> RESERVA | Fechas reales/previstas, estado y noches cobradas. |
| ESTADIA_HUESPED | (id_estadia, id_huesped) | id_estadia -> ESTADIA;<br>id_huesped -> HUESPED                     | Cada pasajero; tarifa aplicada y costo histórico.  |

## 8.3 Cardinalidades y restricciones

| Relación             | Cardinalidad | Interpretación                                                                       |
|----------------------|--------------|--------------------------------------------------------------------------------------|
| ROL - USUARIO        | 1 : 0..*     | Un usuario tiene exactamente un rol; un rol puede no tener usuarios.                 |
| HABITACION - RESERVA | 1 : 0..*     | Una reserva tiene una habitación; una habitación tiene muchas reservas en el tiempo. |

| Relación                  | Cardinalidad | Interpretación                                                                           |
|---------------------------|--------------|------------------------------------------------------------------------------------------|
| HABITACION - ESTADIA      | 1 : 0..*     | Una estadía tiene una habitación; una habitación conserva distintas estadías históricas. |
| HUESPED - RESERVA         | 1 : 0..*     | Una reserva tiene un titular; un huésped puede titularizar varias reservas.              |
| RESERVA - ESTADIA         | 0..1 : 0..1  | Una reserva origina como máximo una estadía; puede existir estadía sin reserva.          |
| ESTADIA - ESTADIA_HUESPED | 1 : 1..*     | Cada estadía registrada tiene uno o más pasajeros.                                       |
| HUESPED - ESTADIA_HUESPED | 1 : 0..*     | Cada participación identifica un huésped; el huésped puede alojarse varias veces.        |

El modelo lógico usa notación pata de cuervo, claves PK/FK/UQ y tipos de datos propuestos. NULL identifica información todavía no definida: reserva de origen de un ingreso directo, salida real de una estadía activa y resultados de cobro antes de cerrar. El resto de los campos de identificación y referencias obligatorias debe ser NOT NULL. Las fechas deben respetar salida > entrada; capacidad y cantidad son enteros positivos. La pareja de ESTADIA\_HUESPED es única. Capacidad, superposición y mínimo de un pasajero requieren validación conjunta/transaccional: una FK aislada no garantiza esas reglas.

## 8.4 Normalización

Primera forma normal: cada celda almacena un valor. Los pasajeros de una estadía no se guardan en una lista dentro de HABITACION; cada participación es una fila de ESTADIA\_HUESPED.

Segunda forma normal: las tablas de identificación tienen claves simples. En la tabla de unión, los datos personales dependen de id\_huesped y se mantienen en HUESPED; fechas y noches comunes dependen de id\_estadia y se mantienen en ESTADIA. La tarifa aplicada y el costo histórico se vinculan a la participación completa.

Tercera forma normal: nombre del rol está en ROL; capacidad y orientación están en HABITACION; nombres del titular están en HUESPED. Esos datos no se repiten en RESERVA o ESTADIA. La ocupación y el total de cuenta se derivan, en lugar de duplicarlos como fuentes de verdad. El costo individual se conserva como resultado histórico de una regla de cobro aplicada; la tarifa de referencia no es una tabla de precios del hotel. La regla real podría requerir otra estructura una vez validada.

## 9. Sketch, wireframes, mockups y prototipo

Sketch: distribución inicial de cabecera, navegación, indicadores, formulario y resultados. Wireframes: nueve pantallas con campos, controles, listados y acciones sin depender del estilo visual. Mockups: las mismas nueve pantallas con identidad gráfica y ejemplos de datos. Prototipo: navegación y operaciones con persistencia local y validaciones del modelo.

| Pantalla       | Datos y acciones principales                                                | Casos de uso     |
|----------------|-----------------------------------------------------------------------------|------------------|
| Acceso         | Selección de usuario demo y perfil asignado.                                | CU01             |
| Disponibilidad | Fechas, cantidad, capacidad y habitaciones compatibles; ocupación actual.   | CU05             |
| Habitaciones   | Número, capacidad, orientación; registrar y editar.                         | CU02             |
| Huéspedes      | Identificación, nombres y apellidos; registro y selección posterior.        | CU03             |
| Reservas       | Titular, habitación, fechas, cantidad; registrar, modificar, cancelar.      | CU09, CU11, CU12 |
| Check-in       | Reserva o ingreso directo; habitación, fechas y cada pasajero.              | CU04             |
| Check-out      | Estadía, salida, tarifa ficticia, cálculo individual, total y confirmación. | CU06, CU07       |
| Informes       | Período, ocupación y reservas con estados.                                  | CU10             |
| Usuarios       | Nombre único, perfil y estado; registrar y editar.                          | CU08             |

## 9.1 Archivos editables

Los diagramas y todas las vistas de interfaz tienen archivos nativos .drawio con figuras, textos y conectores unidos a los elementos; no son imágenes pegadas en un lienzo. La galería permite abrir cada archivo directamente en [diagrams.net](https://diagrams.net). interfaces.drawio reúne sketch y las dieciocho vistas de wireframe/mockup. Los enlaces de edición de código usan [github.dev](https://github.dev). El material de mockups menciona distintas plataformas, incluido Cacao, Balsamiq y prototipos HTML/CSS. Se conserva [diagrams.net](https://diagrams.net) porque es el formato del repositorio de referencia. No se afirma haber creado proyectos en plataformas distintas.

## 9.2 Alcance implementado

El prototipo registra y edita habitaciones, identifica huéspedes, gestiona reservas, realiza check-in con o sin reserva, registra cada pasajero, calcula noches a partir de fechas y conserva costos individuales al salir. Controla las operaciones por rol demo y genera informes por período. No implementa autenticación productiva, servidor, base de datos central ni exclusión concurrente entre dispositivos. Estas diferencias se mantienen visibles en la documentación para no presentar una maqueta como un sistema instalado en el hotel.

## 10. Planificación Kanban

El tablero conserva tareas técnicas terminadas, artefactos en revisión y decisiones pendientes. Terminar la implementación no equivale a aprobación del docente. No se inventan integrantes ni plazos. Las decisiones sobre identificación, cobro, permisos, habitación compartida y entorno permanecen por validar; la rúbrica y retroalimentación no fueron recibidas.

## 11. Trazabilidad

| RF   | Casos de uso     | Clases/entidades                | Pantallas          |
|------|------------------|---------------------------------|--------------------|
| RF01 | CU02             | Habitacion                      | habitaciones       |
| RF02 | CU03, CU04       | Huesped,<br>EstadiaHuesped      | huespedes, checkin |
| RF03 | CU05             | Habitacion, Reserva,<br>Estadia | disponibilidad     |
| RF04 | CU04             | Estadia, EstadiaHuesped         | checkin            |
| RF05 | CU06, CU07       | Estadia, EstadiaHuesped         | checkout           |
| RF06 | CU07             | EstadiaHuesped                  | checkout           |
| RF07 | CU01, CU08       | Usuario, Rol                    | acceso, usuarios   |
| RF08 | CU09, CU11, CU12 | Reserva, Habitacion,<br>Huesped | reservas           |
| RF09 | CU10             | Reserva, Estadia,<br>Habitacion | informes           |

### 11.1 Verificación

Las pruebas del motor comprueban acceso restringido, duplicados, capacidad, fechas, solapamiento, intervalos contiguos, modificación/cancelación de reservas, identificación de todos los pasajeros, check-in directo, repetición de operaciones, costos por huésped, liberación de ocupación e informes por fechas. La revisión del modelado comprueba códigos de casos, cobertura de requisitos, conectores, decisiones y cardinalidades. Las pruebas técnicas no reemplazan la aprobación académica ni la validación de reglas del hotel.

## 12. Referencias

Materiales del profesor utilizados: Definición de proyectos(2).docx; UML\_CASOS\_DE\_USO\_2(2).jpg; UML\_CASOS\_DE\_USO(2).jpg; CASOS\_DE\_USO\_como\_iniciar(2).pdf; CASOS\_DE\_USO\_extend\_y\_include(2).pdf; diagrama\_de\_procesos(2).jpg; Diagramas de Clase(2).pdf; GUIA\_diagramas\_de\_clase(2).pdf; MODELO\_E\_R\_NORMALIZACION(1).pdf; MOCKUP\_COMPLETO(2).pdf; Github\_Git\_VSCode(1).docx. Se conservan los originales en fuentes/.

Referencias técnicas de notación y diseño consultadas para la revisión:

1. OMG. UML 2.5.1: <https://www.omg.org/spec/UML/2.5.1>
2. draw.io. Clases UML: <https://www.drawio.com/docs/diagram-types/uml/class-diagrams/>

3. draw.io. Modelo entidad-relación: <https://www.drawio.com/docs/diagram-types/entity-relationship-tables/>
4. Balsamiq. Wireframes: <https://balsamiq.com/blog/what-are-wireframes/>
5. Referencia de organización y prototipo inicial: <https://github.com/Yutre3/diego>

Estas referencias aportan notación y técnicas, no funciones nuevas del negocio. El alcance proviene del tema 6. No se recibieron rúbrica, plantilla IEEE original, PPT/PPTX ni videos; no se simula haberlos consultado.